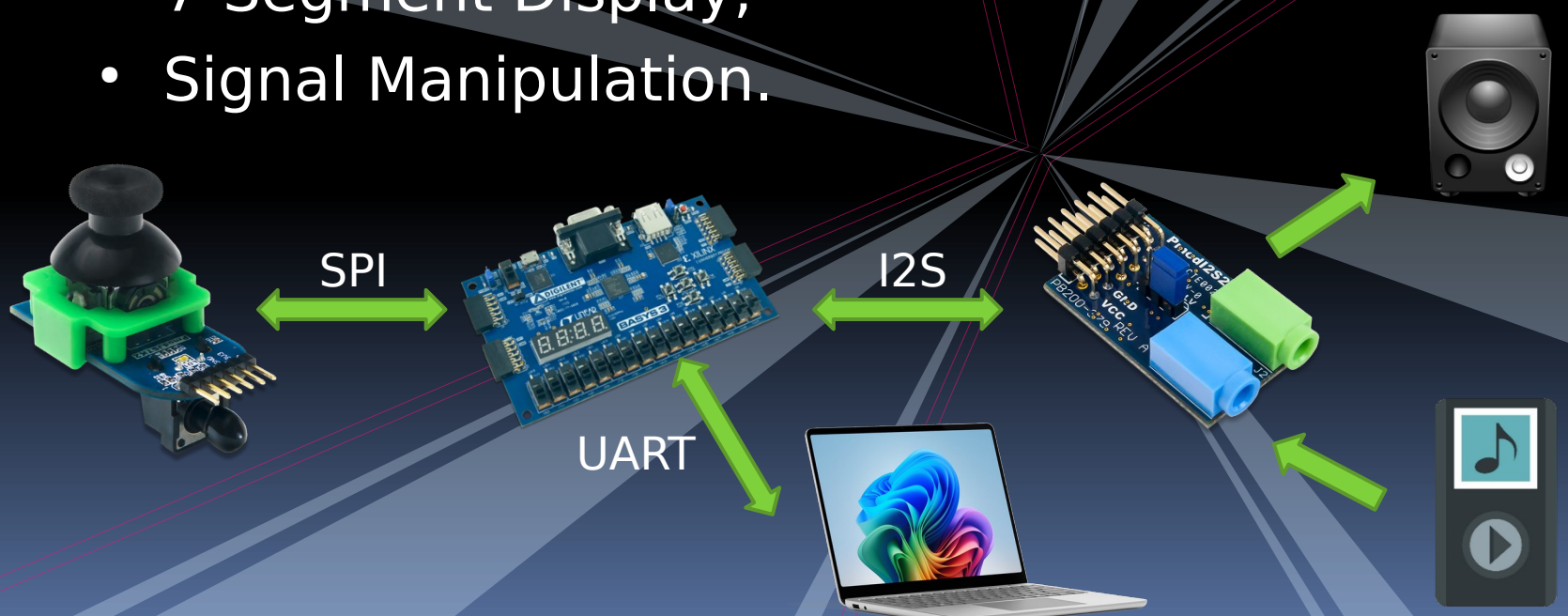
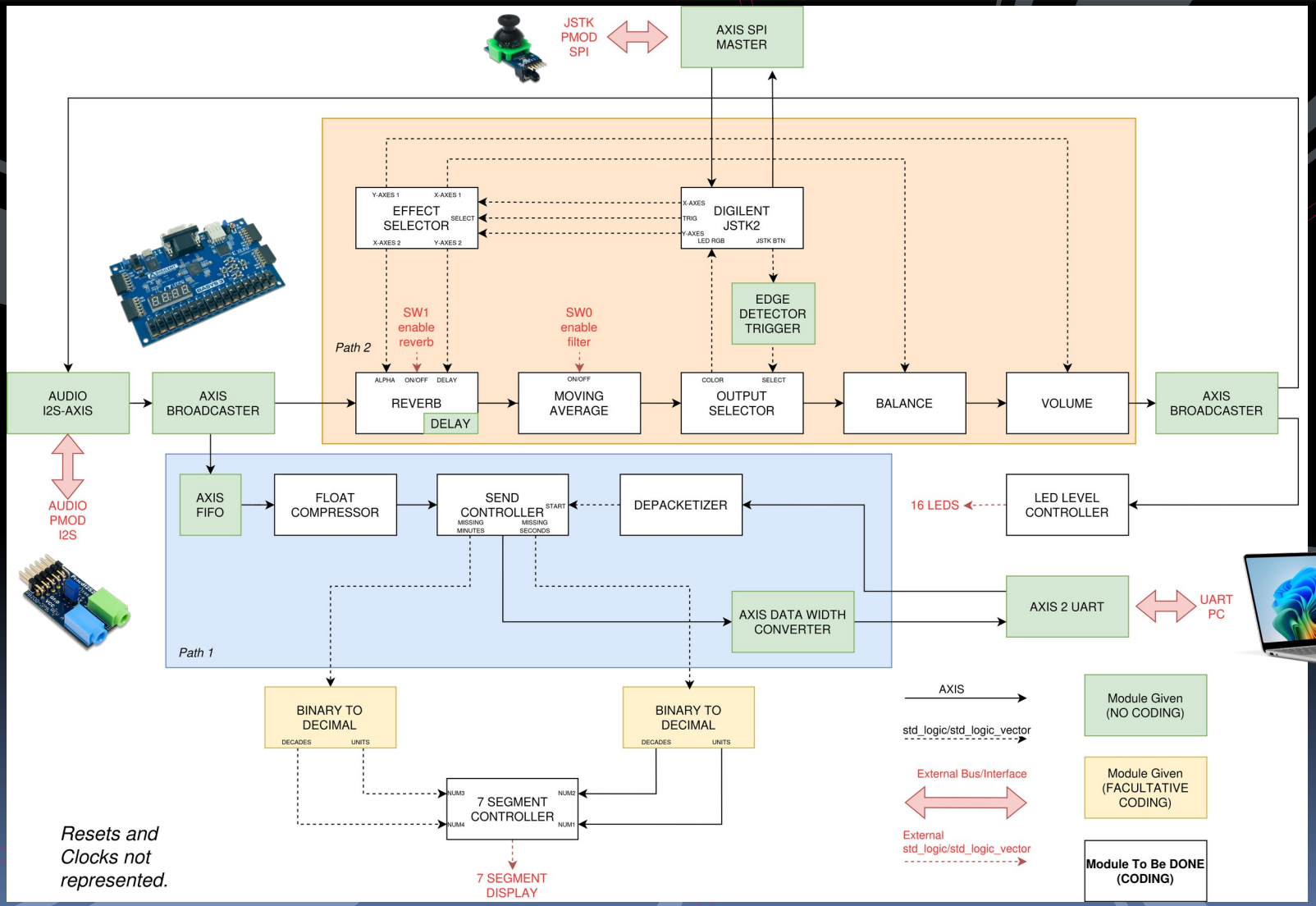
LAB OVERVIEW

Topics

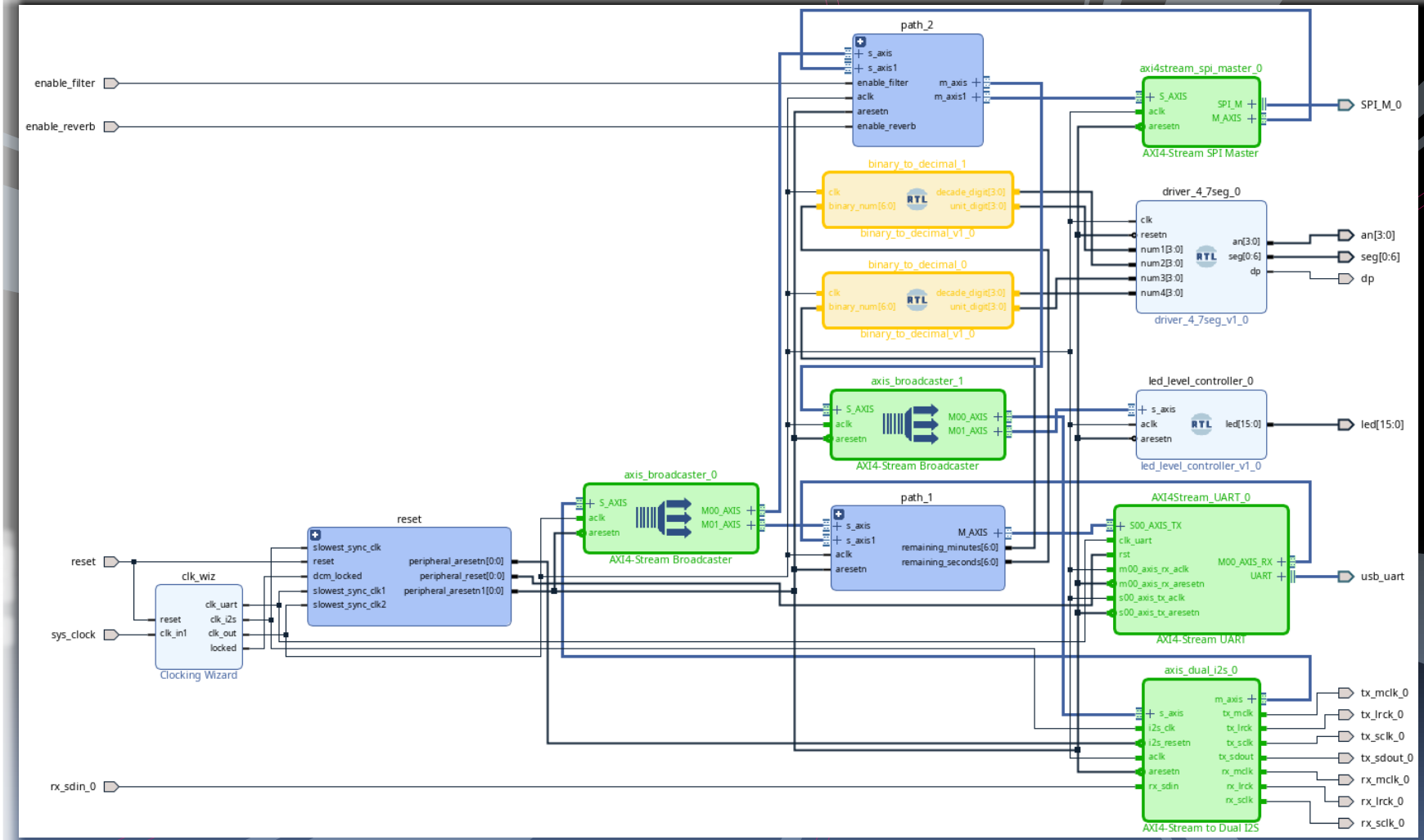
- Digilent Pmod Joystick SPI module,
- Digilent Pmod I2S2 module,
- UART communication between FPGA and PC,
- AXIS,
- 7-Segment Display,
- Signal Manipulation.



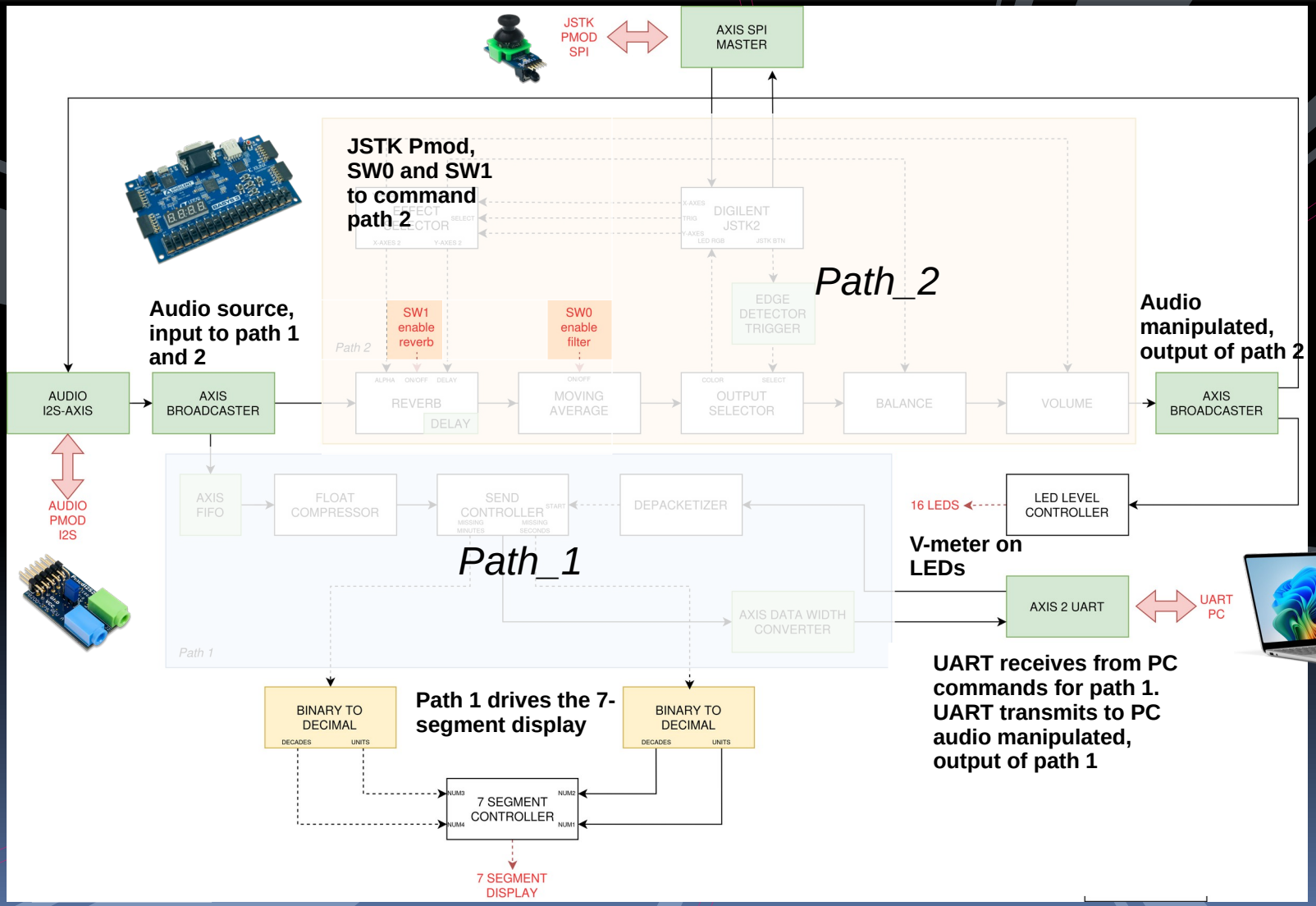
Project Description



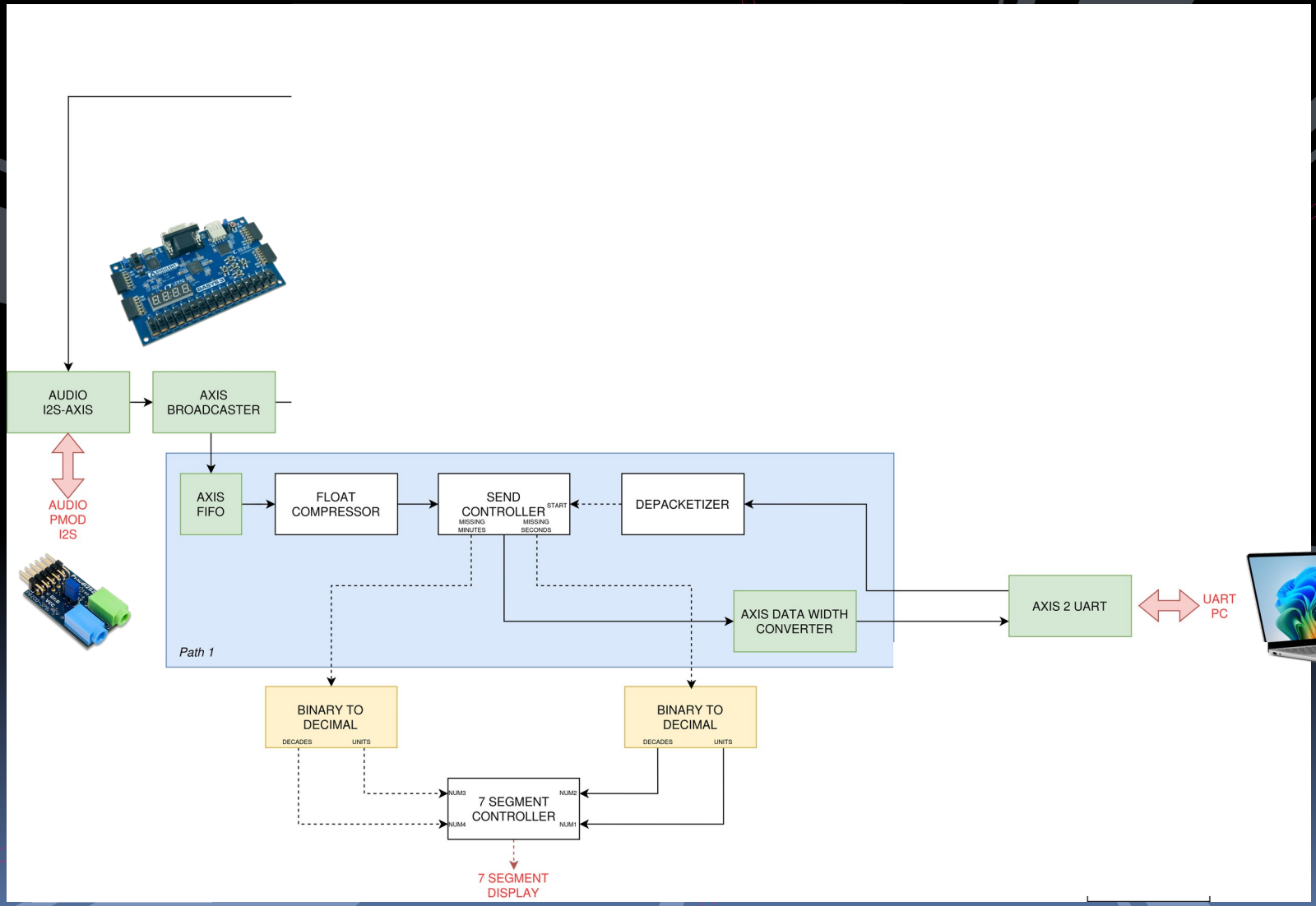
Block Design



Project Description: I/O



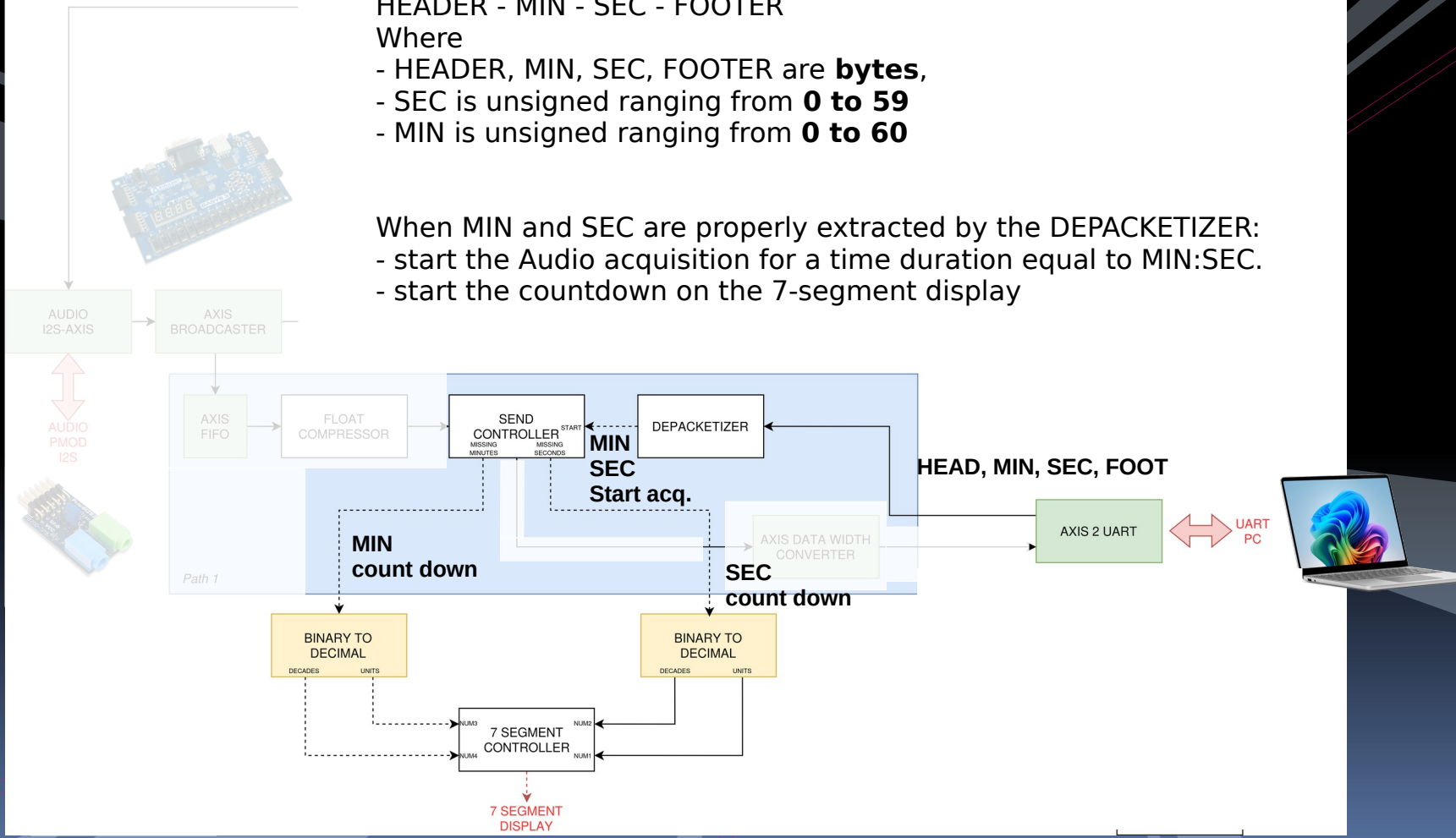
Project Description: Path 1



Path 1: commands

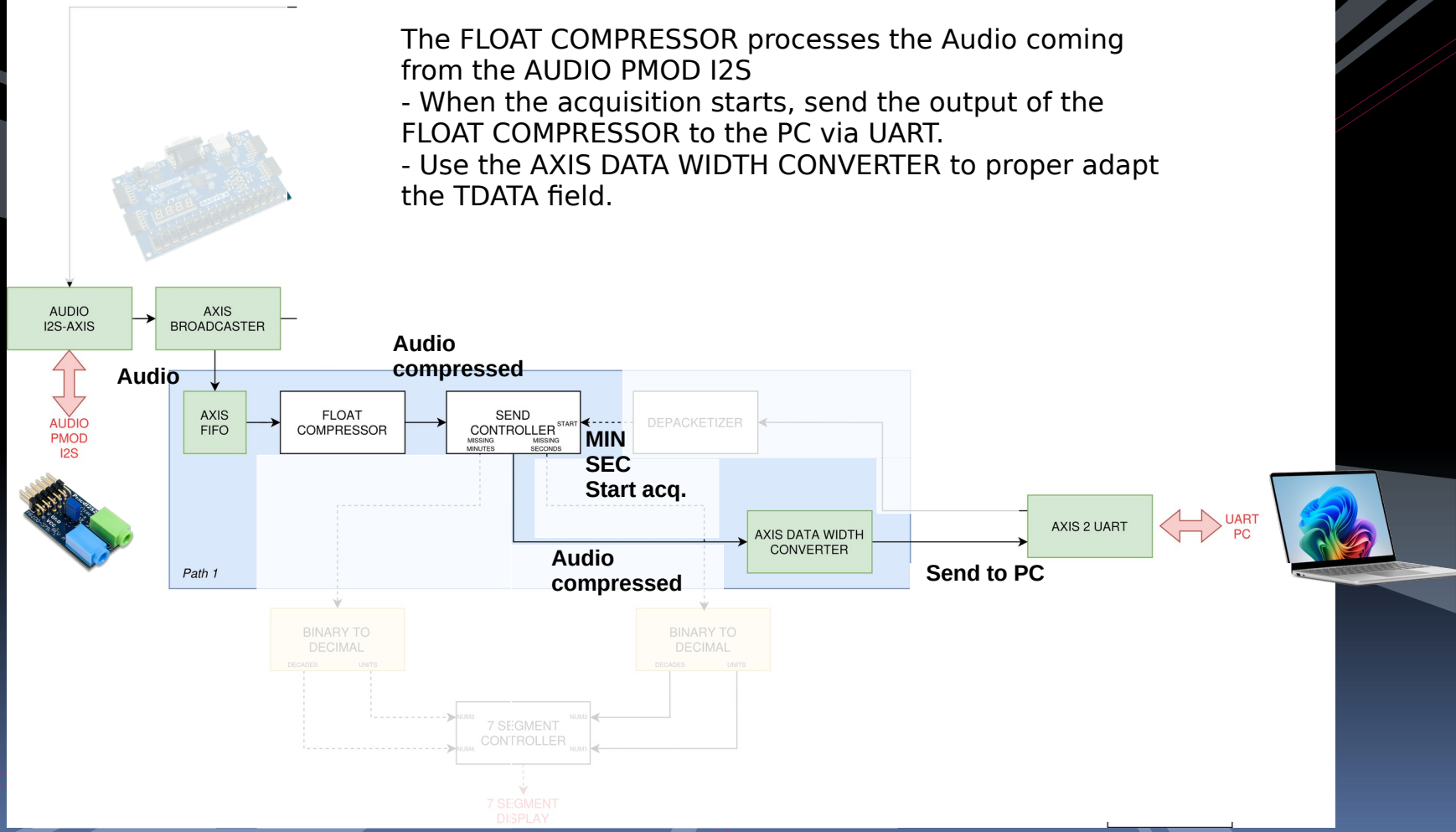
Send, using the PC, the commands via UART with the following format:
HEADER - MIN - SEC - FOOTER
Where
- HEADER, MIN, SEC, FOOTER are **bytes**,
- SEC is unsigned ranging from **0 to 59**
- MIN is unsigned ranging from **0 to 60**

When MIN and SEC are properly extracted by the DEPACKETIZER:
- start the Audio acquisition for a time duration equal to MIN:SEC.
- start the countdown on the 7-segment display

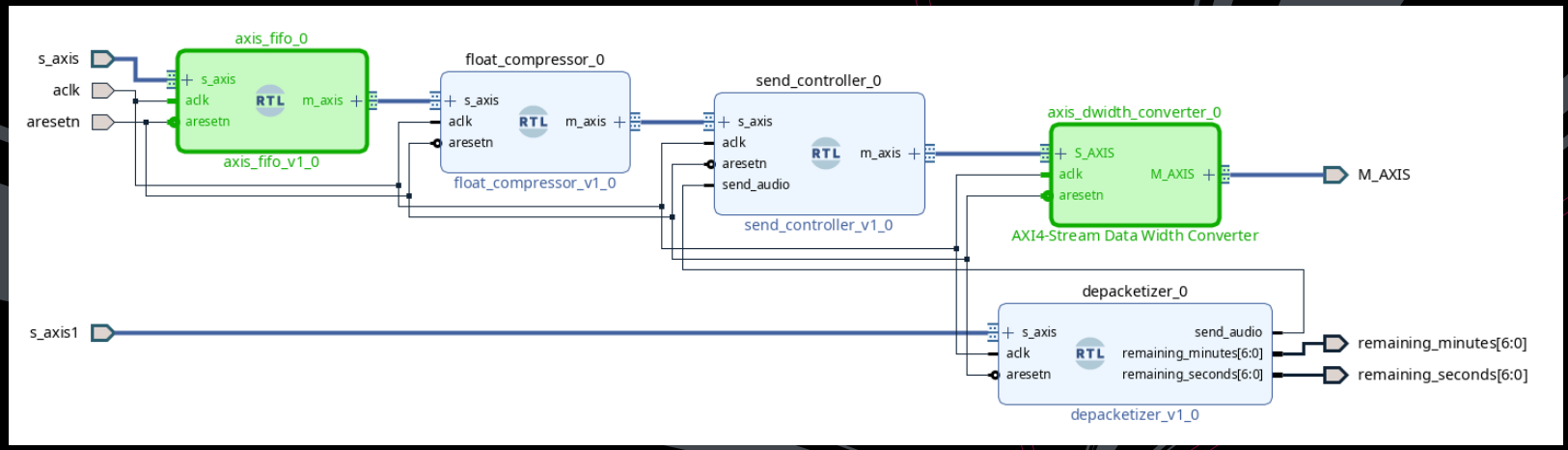


Path 1: acquisition

The FLOAT COMPRESSOR processes the Audio coming from the AUDIO PMOD I2S
- When the acquisition starts, send the output of the FLOAT COMPRESSOR to the PC via UART.
- Use the AXIS DATA WIDTH CONVERTER to proper adapt the TDATA field.

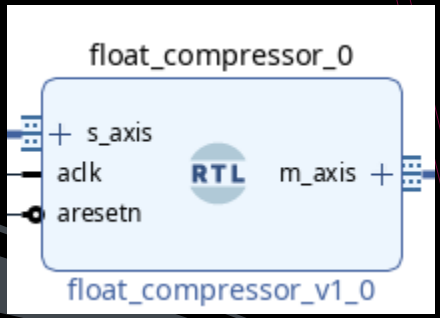


IP Path 1


s_axis
aclk
aresetn
axis_fifo_0
s_axis
aclk
aresetn
m_axis
RTL
axis_fifo_v1_0
float_compressor_0
s_axis
aclk
aresetn
m_axis
RTL
float_compressor_v1_0
send_controller_0
s_axis
aclk
aresetn
send_audio
m_axis
RTL
send_controller_v1_0
axis_dwidth_converter_0
S_AXIS
aclk
aresetn
M_AXIS
AXI4-Stream Data Width Converter
M_AXIS
s_axis1
depacketizer_0
s_axis
aclk
aresetn
send_audio
remaining_minutes[6:0]
remaining_seconds[6:0]
RTL
depacketizer_v1_0
remaining_minutes[6:0]
remaining_seconds[6:0]

IP Path 1: FLOAT COMPRESSOR(1

Compress the signed 24-bit information of TDATA on s_axis interface in a floating-point-like 16-bit TDATA on m_axis interface.

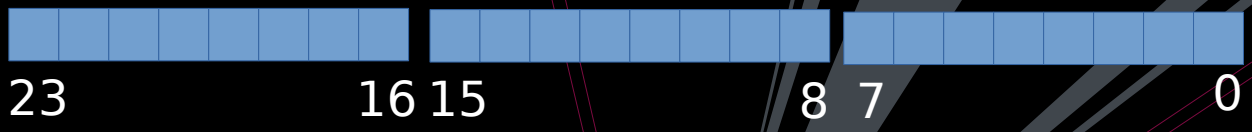


DEFINITION:

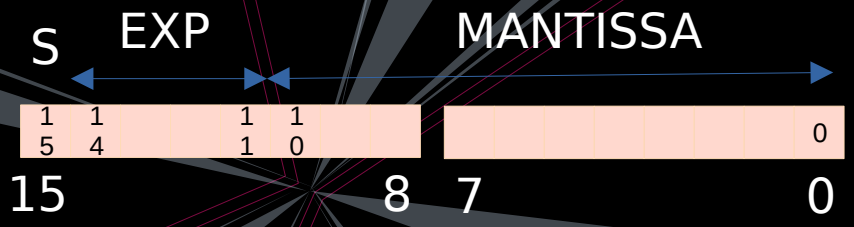
- CHANNEL_LENGHT = 24: TDATA width of s_axis.
- OUTPUT_LENGHT = 16: TDATA width of m_axis.
- EXPONENT_LENGTH = 4: dimension of exponent field in floating-point-like compressed data.
- MANTISSA_LENGTH = OUTPUT_LENGHT-EXPONENT_LENGTH-1= 11: dimension of mantissa field in floating-point-like compressed data.
- SIGN_LENGTH = 1: dimension of sign field in floating-point-like compressed data.

IP Path 1: FLOAT COMPRESSOR(2

signed 24-bit information of TDATA on s_axis



floating-point-like 16-bit TDATA on m_axis

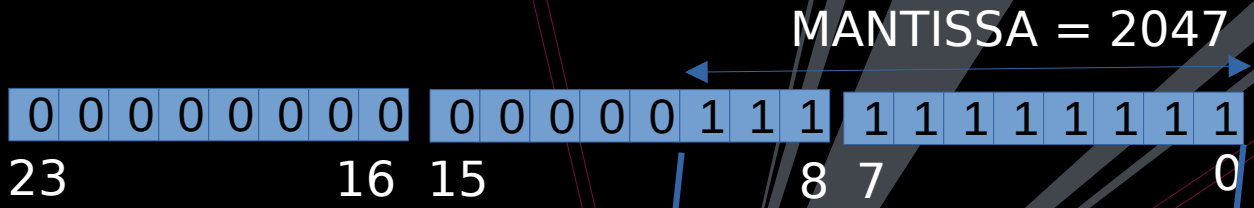


- CHANNEL_LENGHT = 24
- OUTPUT_LENGHT = 16
- EXPONENT_LENGTH = 4
- MANTISSA_LENGTH = 11
- SIGN_LENGTH = 1

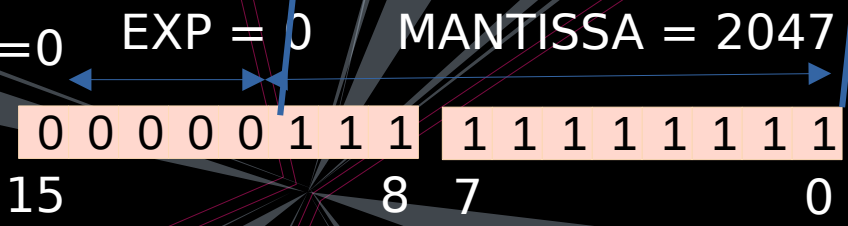
IP Path 1: FLOAT COMPRESSOR(3

Positive number < $2^{MANTISSA_LENGTH}$ = 2048 → example with +2047

signed 24-bit information of TDATA on s_axis



floating-point-like 16-bit TDATA on m_axis



EXP = (unsigned EXPONENT_LENGTH) 0

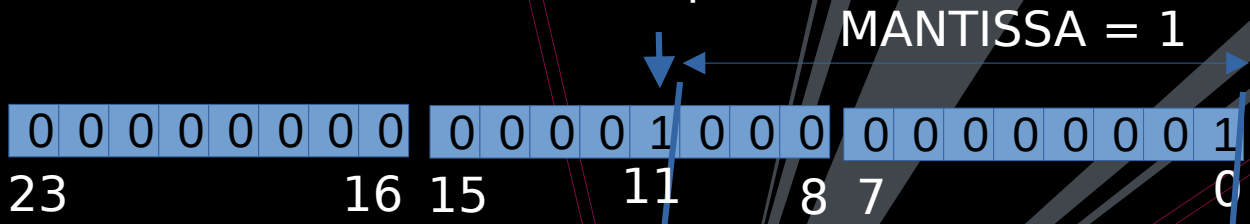
MANTISSA = (unsigned MANTISSA_LENGTH)

- CHANNEL_LENGHT = 24
- OUTPUT_LENGHT = 16
- EXPONENT_LENGTH = 4
- MANTISSA_LENGTH = 11
- SIGN_LENGTH = 1

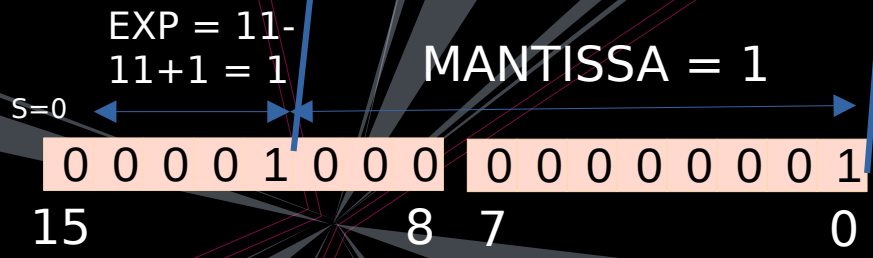
IP Path 1: FLOAT COMPRESSOR(4

Positive number $\geq 2^{MANTISSA_LENGTH} = 2048 \rightarrow$ example with +2049

signed 24-bit information of TDATA on s_axis



floating-point-like 16-bit TDATA on m_axis



EXP = (unsigned EXPONENT_LENGTH) Position of the first “1” starting from MSB – MANTISSA_LENGTH + 1

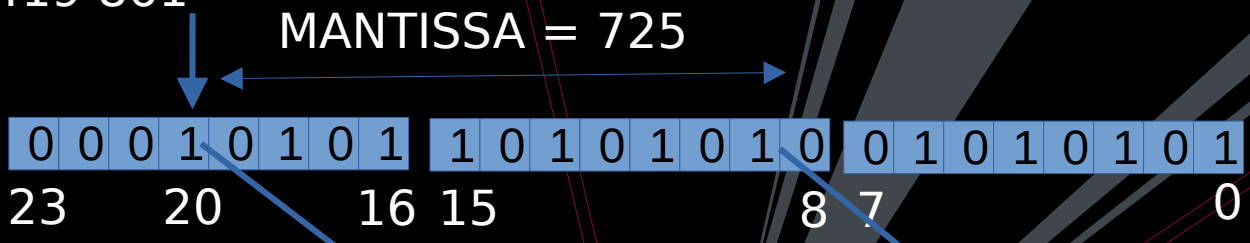
MANTISSA = (unsigned MANTISSA_LENGTH)

- CHANNEL_LENGHT = 24
- OUTPUT_LENGHT = 16
- EXPONENT_LENGTH = 4
- MANTISSA_LENGTH = 11
- SIGN_LENGTH = 1

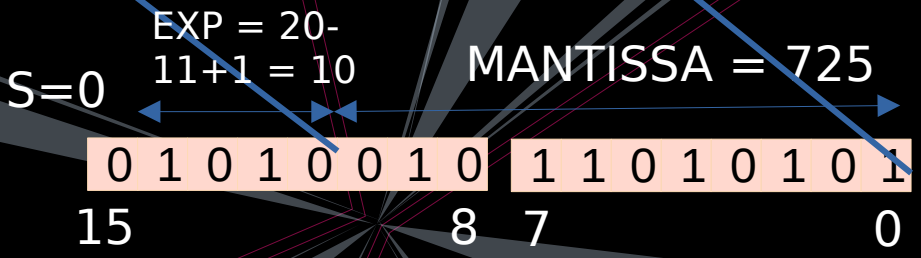
IP Path 1: FLOAT COMPRESSOR(5

example with +1’419’861

signed 24-bit information of TDATA on s_axis



floating-point-like 16-bit TDATA on m_axis



EXP = (unsigned EXPONENT_LENGTH) Position of the first “1” starting from MSB – MANTISSA_LENGTH + 1

MANTISSA = (unsigned MANTISSA_LENGTH)

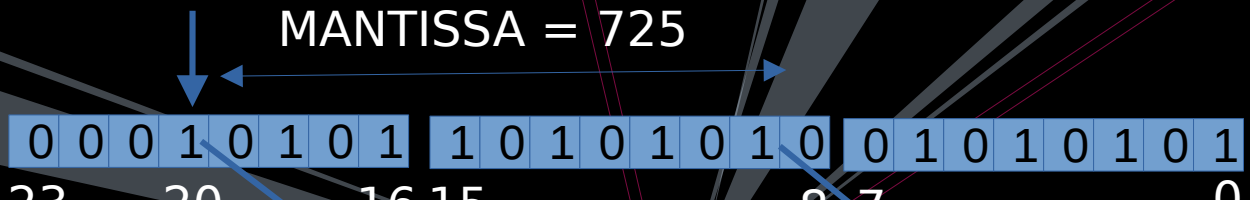
- CHANNEL_LENGHT = 24
- OUTPUT_LENGHT = 16
- EXPONENT_LENGTH = 4
- MANTISSA_LENGTH = 11
- SIGN_LENGTH = 1

IP Path 1: FLOAT COMPRESSOR(6

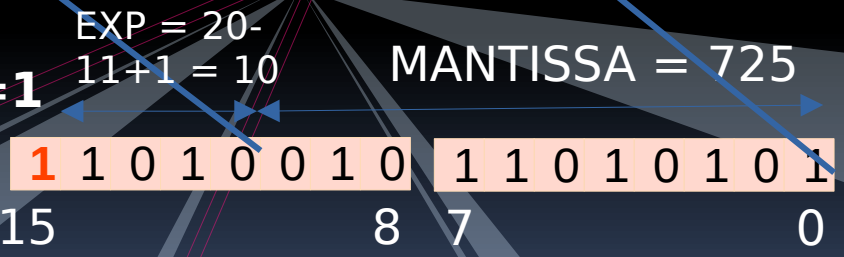
For negative number, perform the module and perform as before.
NB put sign = 1. NBB -0 does not exist.

example with -1’419’861 => |-1’419’861| = 1’419’861

signed 24-bit information of TDATA on s_axis



floating-point-like 16-bit TDATA on m_axis



- CHANNEL_LENGHT = 24
- OUTPUT_LENGHT = 16
- EXPONENT_LENGTH = 4
- MANTISSA_LENGTH = 11
- SIGN_LENGTH = 1

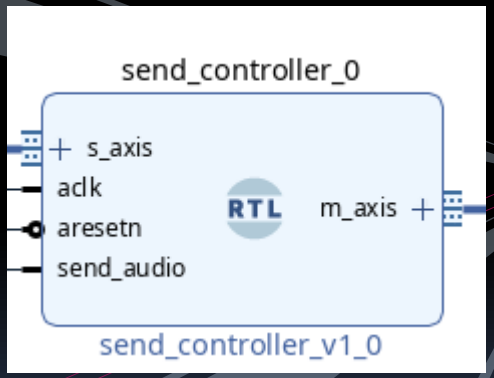
EXP = (unsigned EXPONENT_LENGTH) Position of the first “1” starting from MSB – MANTISSA_LENGTH + 1

MANTISSA = (unsigned MANTISSA_LENGTH)

IP Path 1: SEND CONTROLLER

If send_audio = 1 the audio is sent via the m_axis interface to the S_AXIS_TX of AXI4Stream_UART otherwise are discarded.

Attention consider that the audio is encoded as a packet delimited by TLAST over the axis protocol.



IP Path 1: DEPACKETIZER

Read the M_AXIS_RX of AXI4Stream_UART from the s_axis interface and decodes the UART packet (HEADER = 0x69, MIN (0 to 60), SEC (0 to 59), FOOTER = 0x42 both bytes).

After decoding, the acquisition starts by forcing send_audio = 1 and starting the countdown (remaning_minutes/seconds). The acquisition naturally ends when the countdown reaches 0 minute and 0 seconds.

- Packets with MIN > 60 or SEC > 59 are ignored.

- Packets sent with send_audio = 1 are ignored, exception done for MIN = 0x00 and SEC = 0x00 that **stops** the acquisition and clear the countdown.

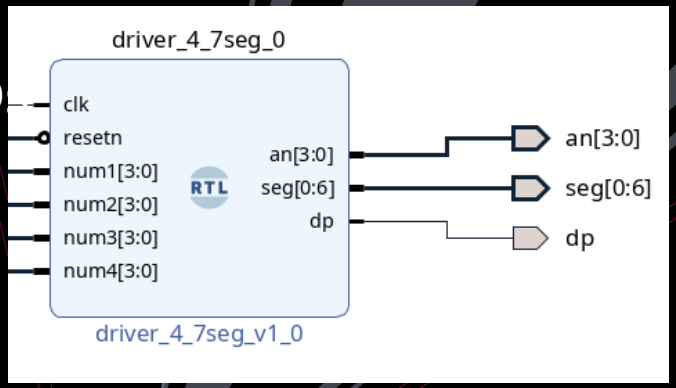
- Packets sent with MIN = 0x00 and SEC = 0x00 when send_audio = 0 are ignored.

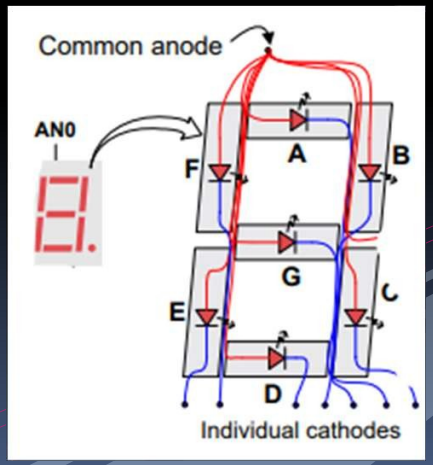


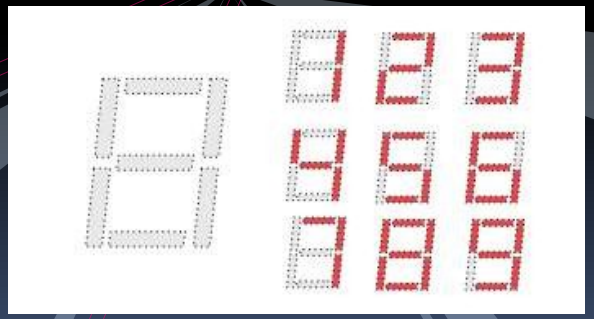
IP Path 1: 7SEGMENT CTRL 1)

A 7-Segment display is composed by 7 LEDs with a common Anode for the 7 LED and 7 individual Cathodes (1-per-LED)

We ignore the “dot”.

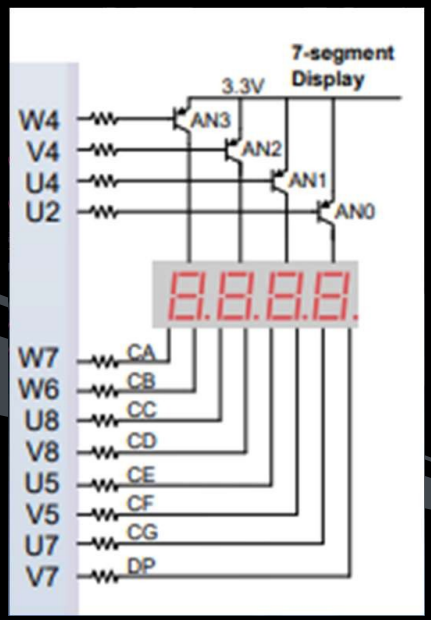




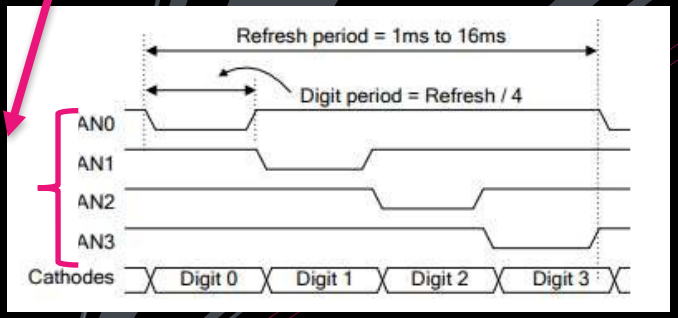


IP Path 1: 7SEGMENT CTRL 2)

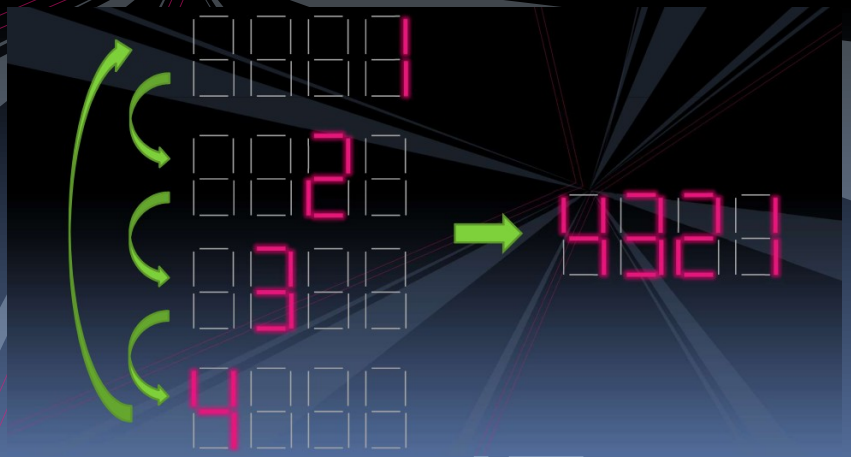
If we have more than one display, we have to multiplex the Digits and the selection to minimize the Outputs.



Warning: "enabled" when LOW

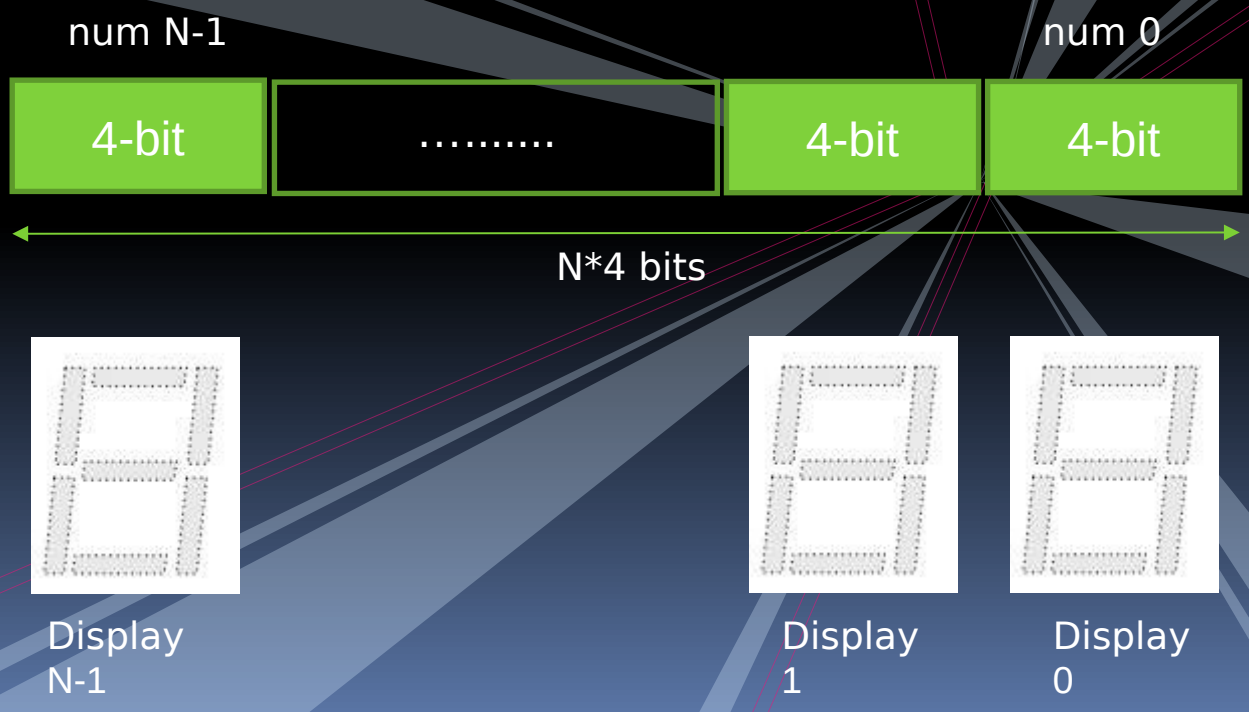


If we multiplex the digits fast enough, they appear all turned on.



IP Path 1: 7SEGMENT CTRL 3)

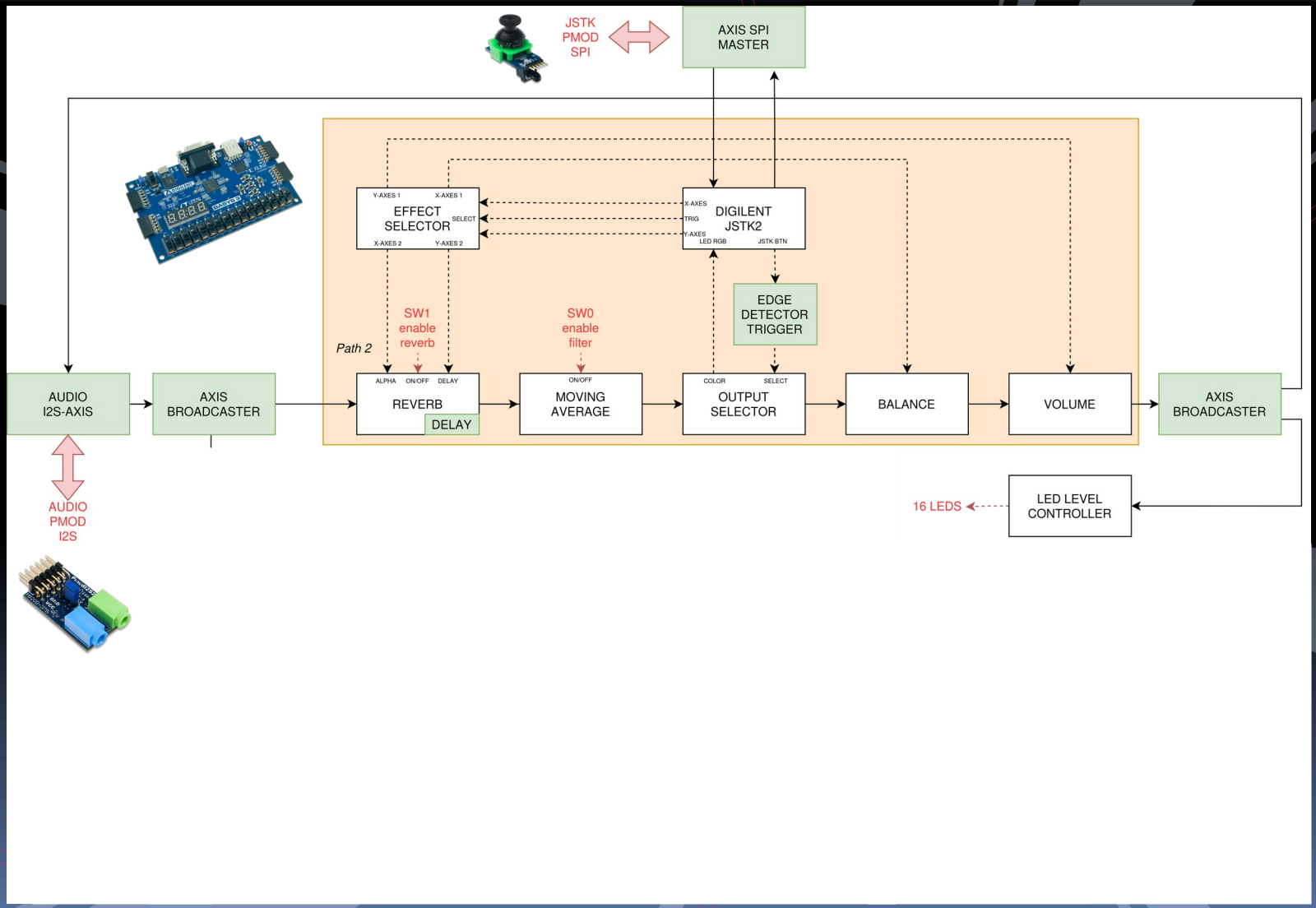
The nibbles coming from driving module are multiplexed one-by-one on the 7-Segments Display.



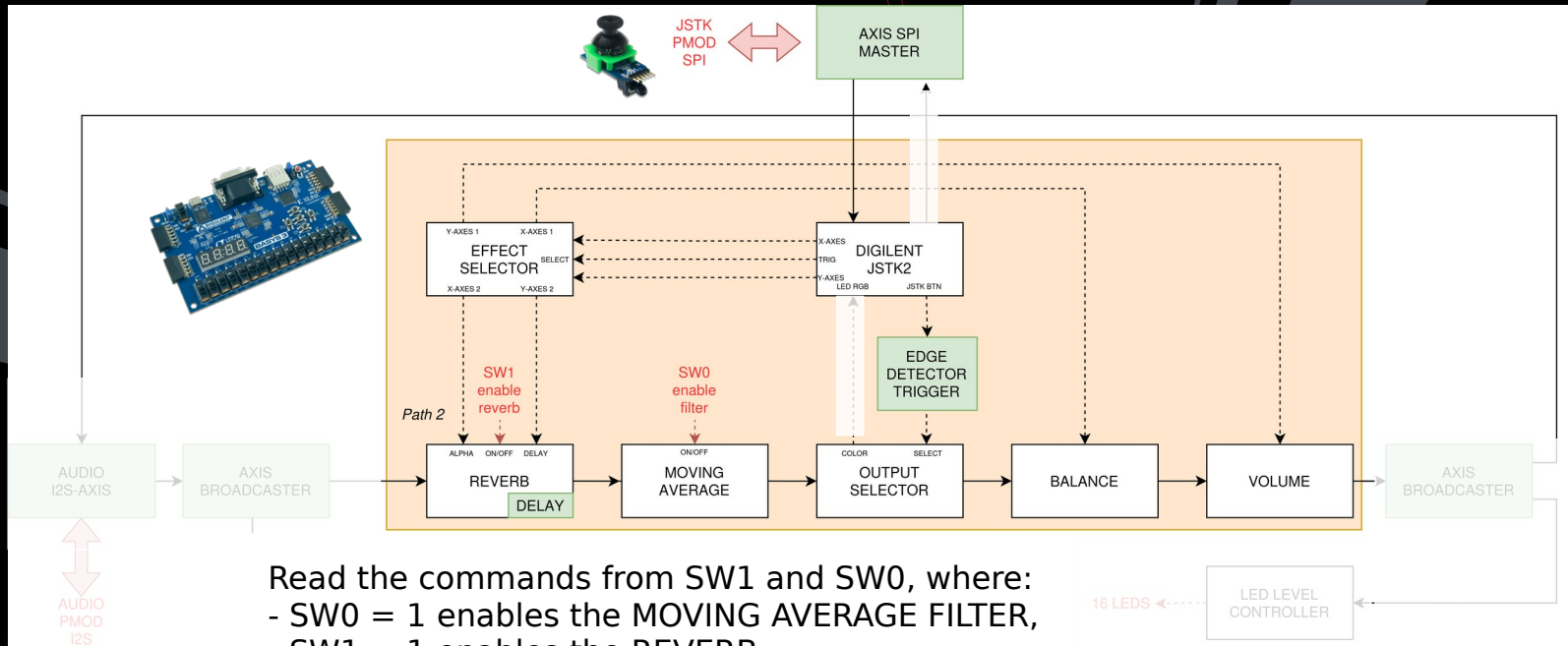
IP Path 1: 7SEGMENT CTRL 4)

```
with num select seg_s <=
"0000001" when "0000",
"1001111" when "0001",
"0010010" when "0010",
"0000110" when "0011",
"1001100" when "0100",
"0100100" when "0101",
"0100000" when "0110",
"0001111" when "0111",
"0000000" when "1000",
"0000100" when "1001",
"0001000" when "1010",
"1100000" when "1011",
"0110001" when "1100",
"1000010" when "1101",
"0110000" when "1110",
"0111000" when "1111",
"1111111" when others;
```

Path 2


JSTK
PMOD
SPI
AXIS SPI
MASTER
EFFECT
SELECTOR
DIGILENT
JSTK2
EDGE
DETECTOR
TRIGGER
SW1
enable
reverb
SW0
enable
filter
Path 2
AUDIO
I2S-AXIS
AXIS
BROADCASTER
REVERB
DELAY
MOVING
AVERAGE
OUTPUT
SELECTOR
BALANCE
VOLUME
AXIS
BROADCASTER
AUDIO
PMOD
I2S
16 LEDS
LED LEVEL
CONTROLLER

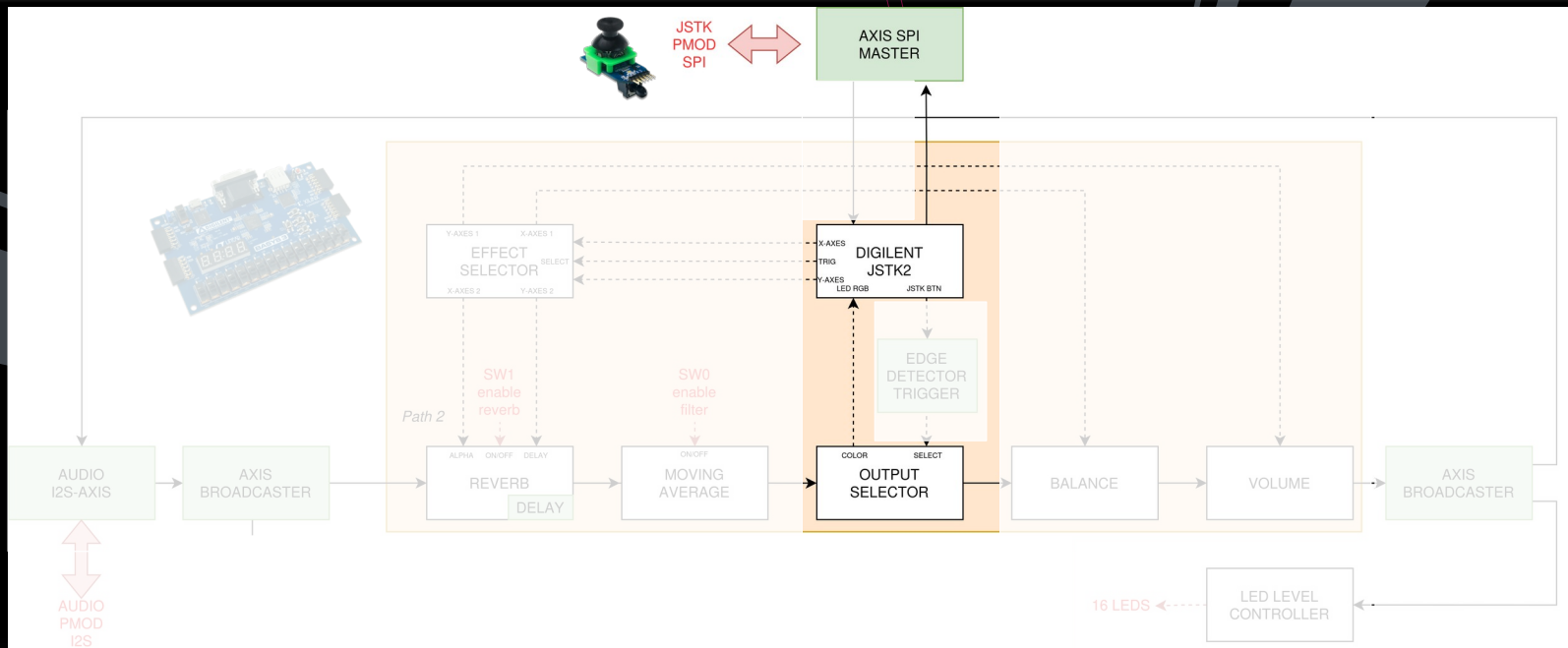
Path 2: commands



Read the commands from SW1 and SW0, where:
- SW0 = 1 enables the MOVING AVERAGE FILTER,
- SW1 = 1 enables the REVERB.

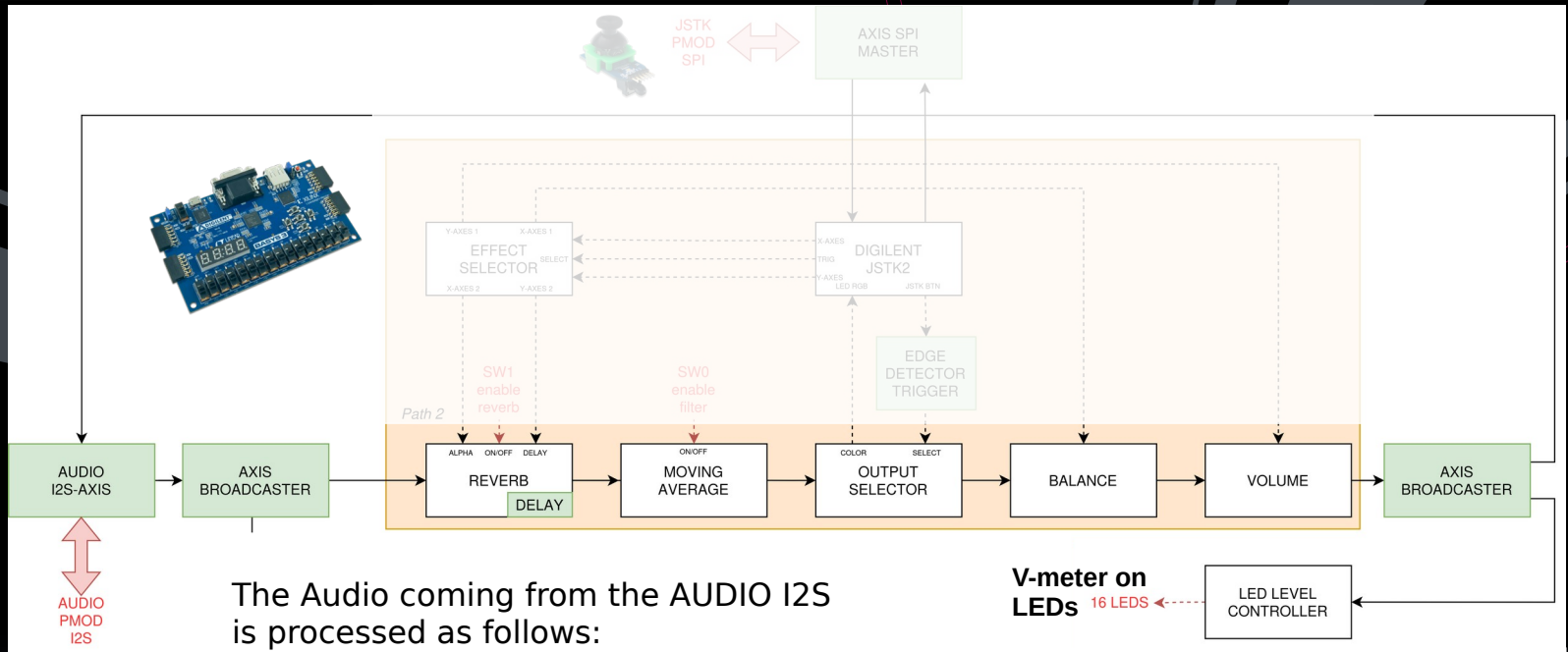
Read the commands from JSTK, where:
- JSTK X-AXIS = ALPHA (or gain) of REVERB if JSTK TRIG = 1,
- JSTK Y-AXIS = DELAY of REVERB if JSTK TRIG = 1,
- JSTK X-AXIS = BALANCE of BALANCE if JSTK TRIG = 0,
- JSTK Y-AXIS = VOLUME of VOLUME if JSTK TRIG = 0,
- A rising edge of JSTK BTN, output of EDGE DETECTION TRIGGER, change the behavior of OUTPUT SELECTOR.

Path 2: status



If the behavior of OUTPUT SELECTOR changes, communicate the status of OUTPUT SELECTOR by changing the color of the RGB LED available on JSTK PMOD SPI.

Path 2: processing

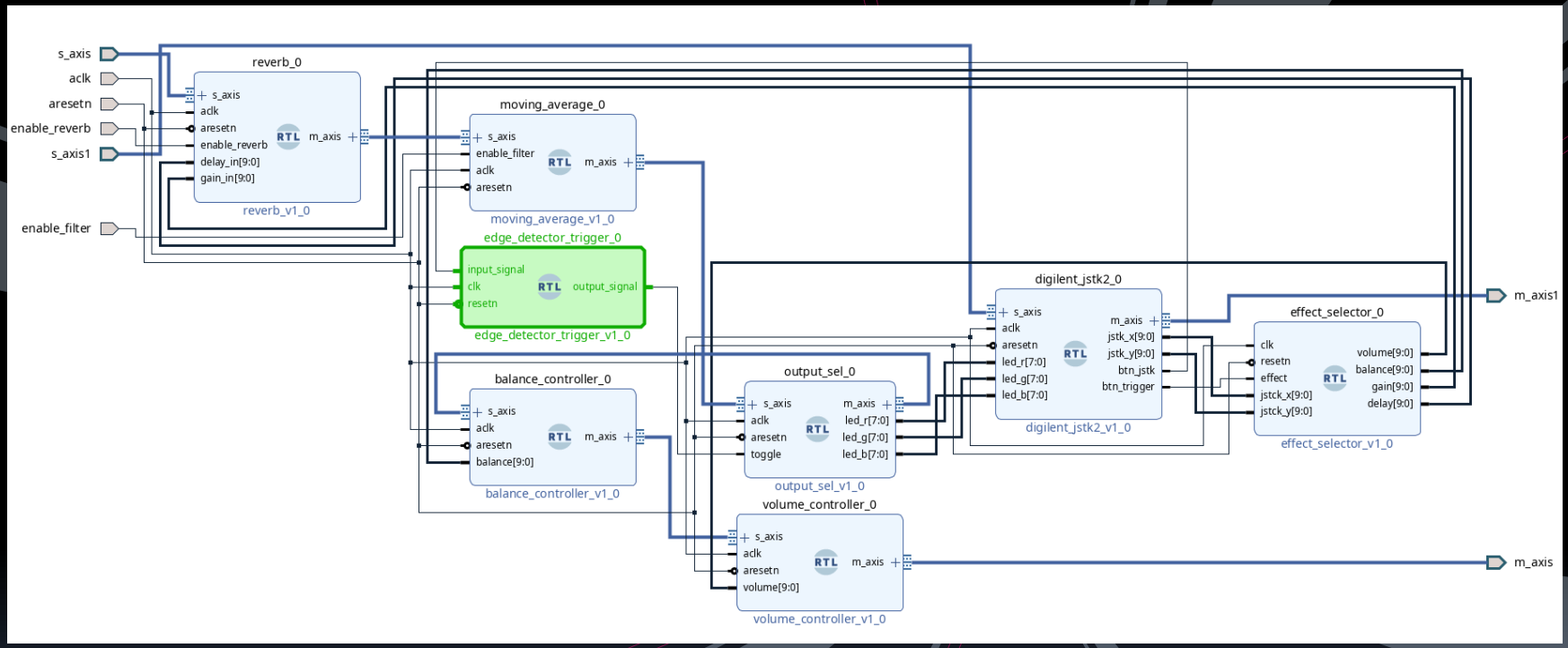


The Audio coming from the AUDIO I2S is processed as follows:

- A reverb is added by REVERB
- A low-pass filtering is performed by MOVING AVERAGE,
- A proper audio processing on left and right channel is done by OUTPUT SELECTOR,
- A balance between left and right channels and audio tuning are implemented by the BALANCE and VOLUME.

The audio in output of Path 2 is send
- to the AUDIO I2S and
- to the LED LEVEL CONTROLLER

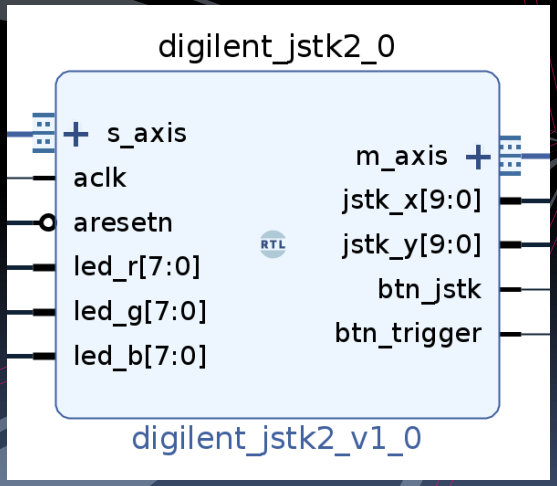
IP Path 2


s_axis
aclk
aresetn
enable_reverb
s_axis1
enable_filter
reverb_0
s_axis
aclk
aresetn
enable_reverb
delay_in[9:0]
gain_in[9:0]
RTL
m_axis
reverb_v1_0
moving_average_0
s_axis
enable_filter
aclk
aresetn
RTL
m_axis
moving_average_v1_0
edge_detector_trigger_0
input_signal
clk
resetn
RTL
output_signal
edge_detector_trigger_v1_0
balance_controller_0
s_axis
aclk
aresetn
balance[9:0]
RTL
m_axis
balance_controller_v1_0
output_sel_0
s_axis
aclk
aresetn
toggle
RTL
m_axis
led_r[7:0]
led_g[7:0]
led_b[7:0]
output_sel_v1_0
volume_controller_0
s_axis
aclk
aresetn
volume[9:0]
RTL
m_axis
volume_controller_v1_0
digilent_jstk2_0
s_axis
aclk
aresetn
led_r[7:0]
led_g[7:0]
led_b[7:0]
RTL
m_axis
jstk_x[9:0]
jstk_y[9:0]
btn_jstk
btn_trigger
digilent_jstk2_v1_0
effect_selector_0
clk
resetn
effect
jstck_x[9:0]
jstck_y[9:0]
RTL
volume[9:0]
balance[9:0]
gain[9:0]
delay[9:0]
effect_selector_v1_0
m_axis1
m_axis

IP Path 2: DIGILENT JSTK2

- Send via the m_axis interface to the S_AXIS of axis4stream_spi_master the led_r, led_g, and led_b ports.

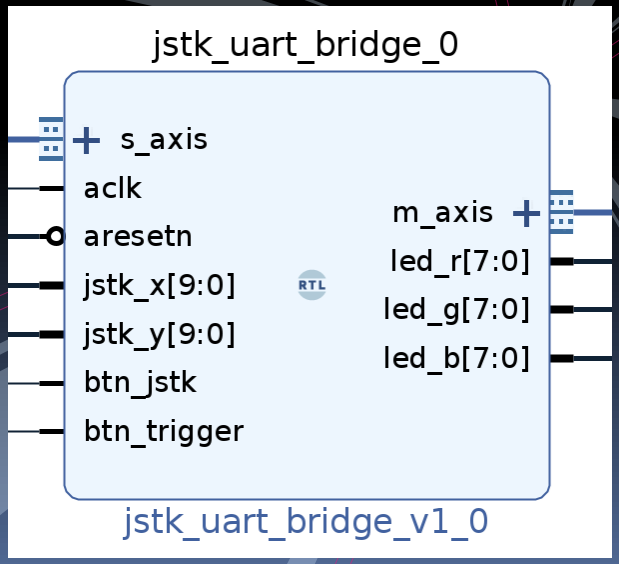
- Read the M_AXIS of axis4stream_spi_master from the s_axis interface and extract jstk_x, jstj_y, btn_jstk, and btn_trigger.



* For further details see the PMOD JSTK module.

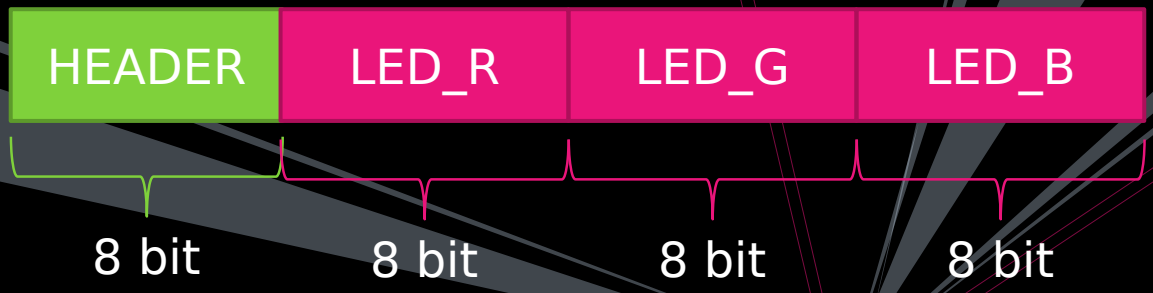
Check DIGILENT JSTK2

To test your digilent_jstk2 interface you can use this jstk_uart_bridge (provided), to send and receive those values to the UART IP-Core and see them via a serial interface

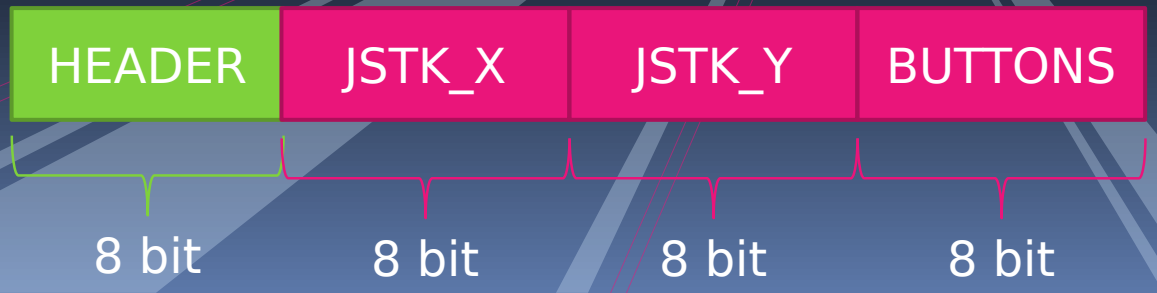


Check DIGILENT JSTK2

You need to send a 4-byte Packet from the serial terminal composed as follows:

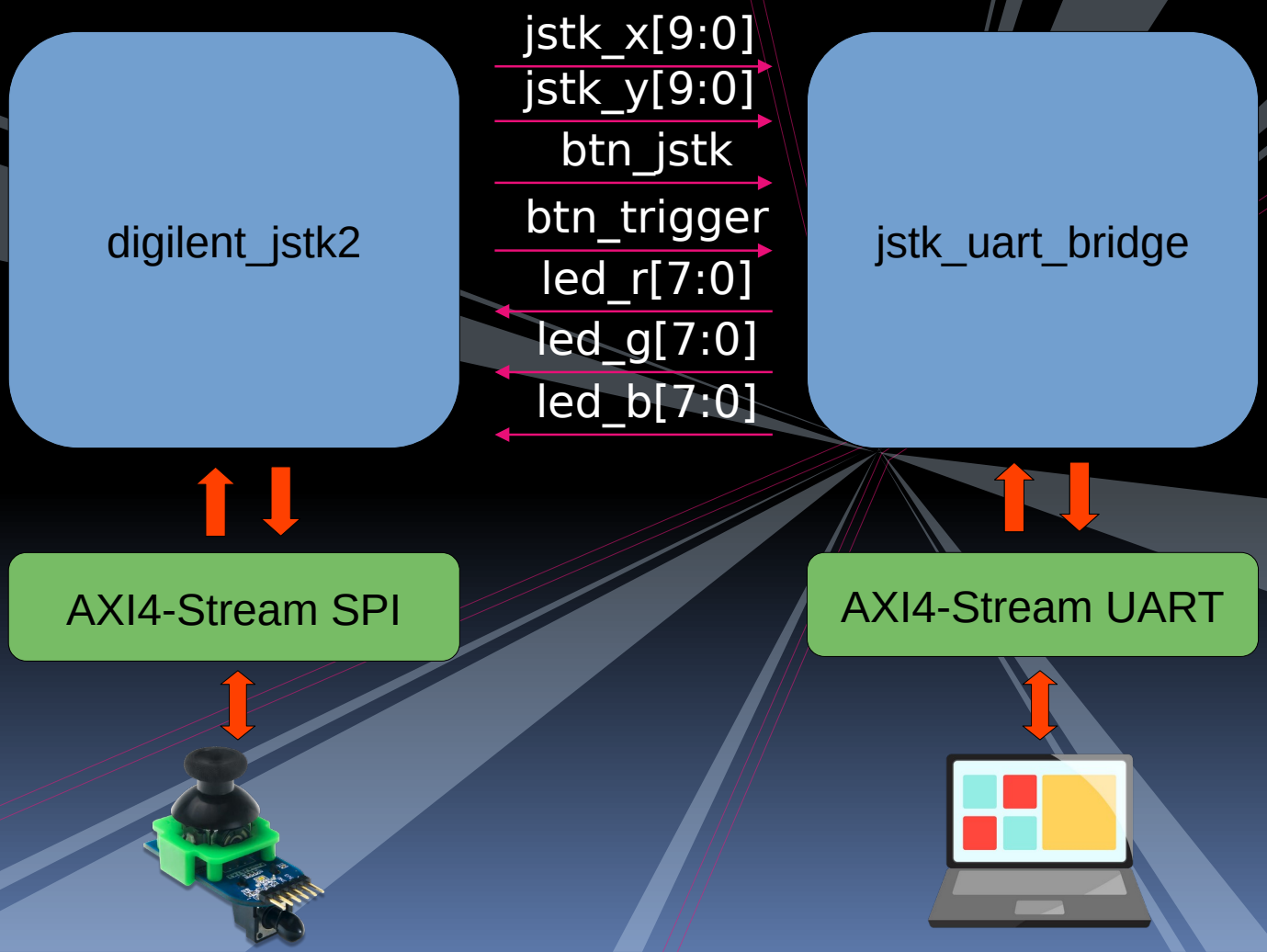


You'll receive back (if everything works fine) packets like this (only the X/Y MSBs are sent by the J_UART_BRIDGE):



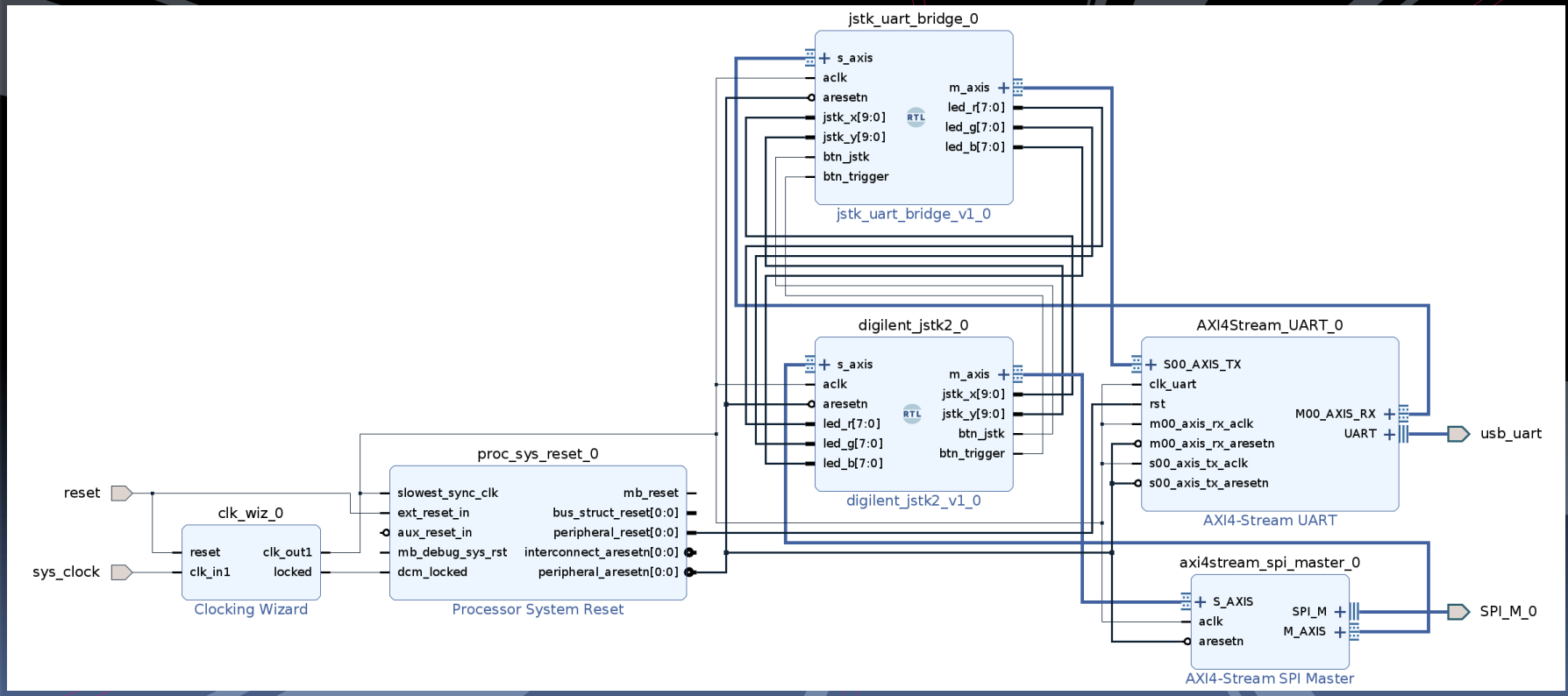
Check DIGILENT JSTK2

At the end, your test project should have this struc



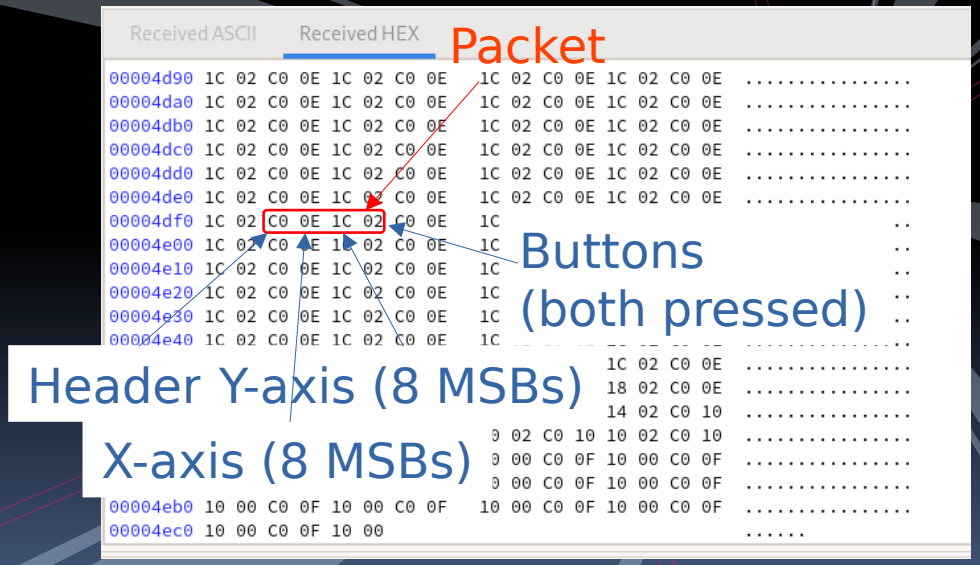
Check DIGILENT JSTK2

At the end, your test project should have this structure:



Check DIGILENT JSTK2

Open your serial terminal and see if you receive the expected data when you move the joystick and press the buttons:



Check DIGILENT JSTK2

Send some data in the proper format and see if the LED changes color.

For example:

- `C0 ff ff ff` → white
- `C0 00 00 00` → OFF
- `C0 ff 00 00` → red
- `C0 ff ff 00` → yellow
- `C0 10 00 10` → light purple

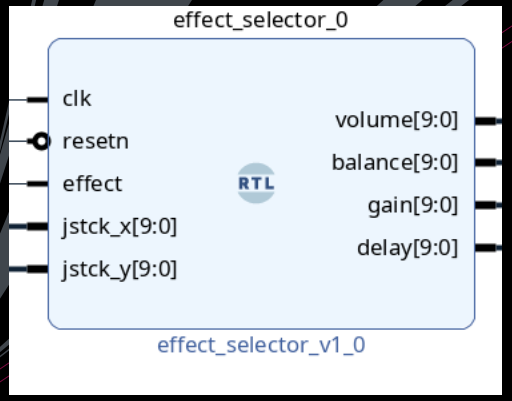
IP Path 2: EFFECT SELECTOR

if JSTK TRIG = 1

- JSTK X-AXIS = ALPHA (or gain) of REVERB
- JSTK Y-AXIS = DELAY of REVERB
- volume and balance stay to the previous selected value (latched).

if JSTK TRIG = 0

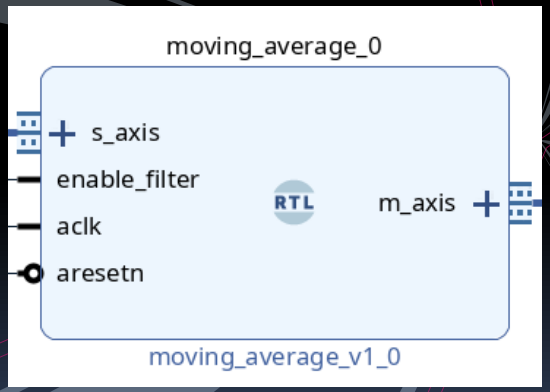
- JSTK X-AXIS = BALANCE of BALANCE
- JSTK Y-AXIS = VOLUME of VOLUME
- ALPHA (or gain) and DELAY stay to the previous selected value (latched).



IP Path 2: MOVING AVERAGE

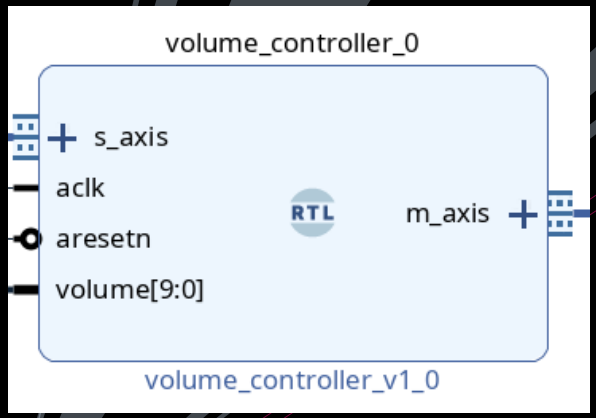
The moving average module must be able to selectively apply a moving average filtering (of a fixed order of 32, set by generic) on the samples.

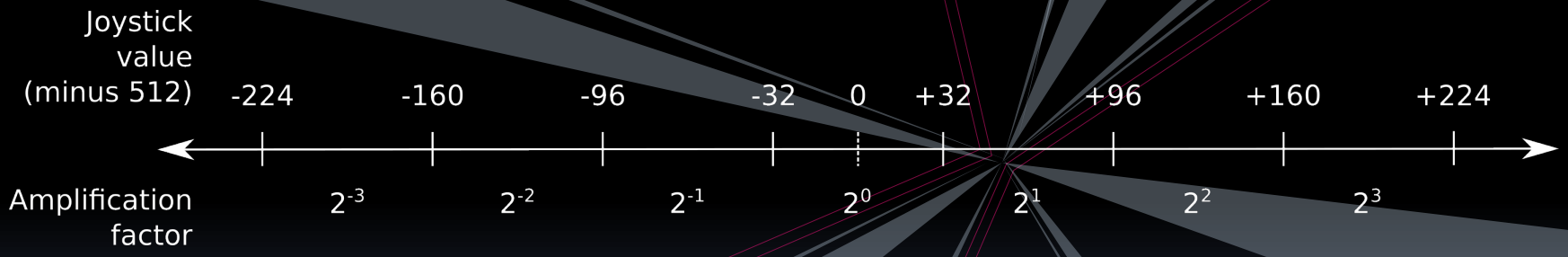
The module should filter the samples when “enable_filter” is high, and should simply pass the samples unmodified when “enable_filter” is low.



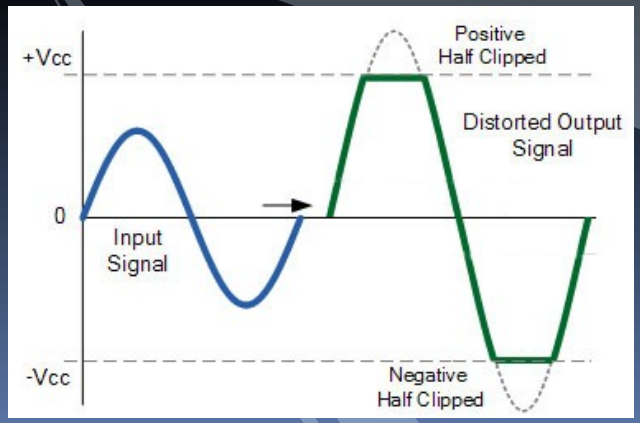
IP Path 2: VOLUME

We perceive “loudness” in a logarithmic way, so the volume control should be exponential. The amplification factor should double every 2^N “joystick units” (with the center in the half of the joystick dynamic, with N as generic).
N=6 returns good results.





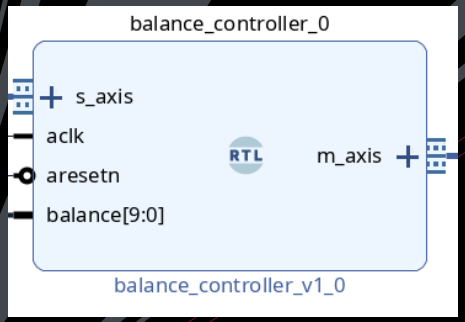
Be careful when multiplying: the signal must saturate at the maximum possible value (“clipping”), you must handle this manually to avoid unexpected results.

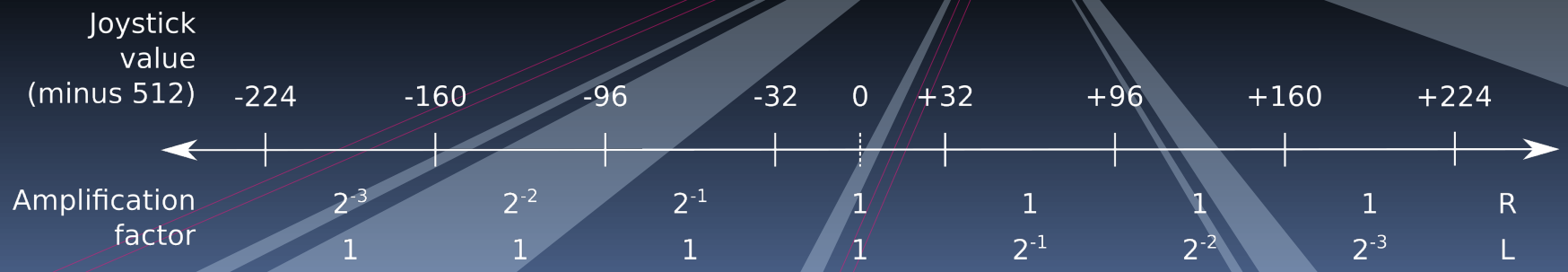


IP Path 2: BALANCE

Use an exponential amplification factor also for the balance control.

- Moving the joystick to the right decreases the left channel volume.
- Moving the joystick to the left decreases the right channel volume.

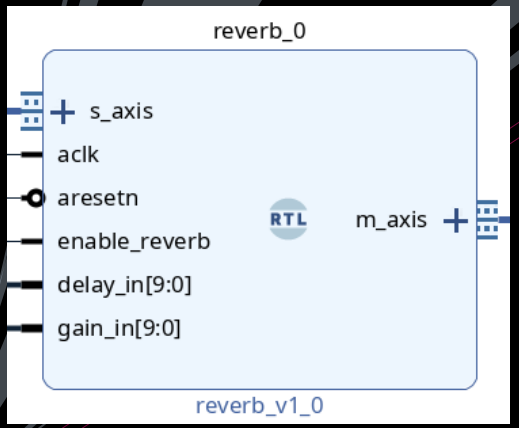




IP Path 2: REVERB 1)

Perform the following formula on both channels (left and rigth), with the following order:

- mul_res = gain_in * y[n - delay_in]
- data_gain = mul_res/2^{gain_length}
- sum_res = x[n] + data_gain
- y[n] = sat(sum_res)



Where:

- x[n] is the TDATA value of the current s_axis transaction.
- y[n] is the current value of TDATA in output, going also into the “delay” module (see next slide)
- y[n - delay_in] is the output of the “delay” module (see next slide)
- sat() is the same saturation algorithm of VOLUME

IP Path 2: REVERB 2)

Delay is described by us by the following module.

```
component delay is
    generic(
        CHANNEL_LENGHT  : integer;
        DELAY_LENGHT  : integer;
        DELAY_INIT : integer;
        LOG2_DELAY_INCR : integer
    );
    Port (

            aclk            : in std_logic;
            aresetn         : in std_logic;

            delay_in        : in std_logic_vector(DELAY_LENGHT-1 downto 0);

            data_in         : in std_logic_vector(CHANNEL_LENGHT-1 downto 0);
            data_in_valid   : in std_logic;

            data_out        : out std_logic_vector(CHANNEL_LENGHT-1 downto 0)
    );
end component;
```

24 bit (dimension of TDATA in Audio I2S)

10 bit (dimension of y-axis in JSTK)

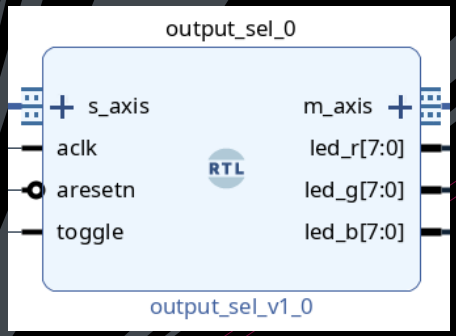
2

IP Path 2: OUTPUT SELECTOR 1)

When toggle = 1 (i.e., pressing of JSKT button) change the behavior moving form status I to status I+1.

- left and right are TDATA of s_axis with TLAST = 0 and 1 respectively.

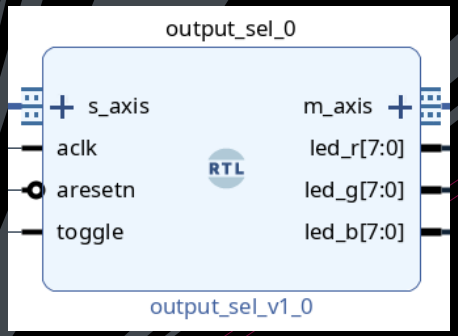
- sat() is the saturation algorithm of VOLUME.



Status	LEFT on m_axis	RIGHT on m_axis
l_r	left	right
mute	0	0
l_l	left	left
r_r	right	right
lpr_lpr	(left+right)/2 or sat(left+right)	(left+right)/2 or sat(left+right)
lmr_lmr	(left-right)/2 or sat(left-right)	(left-right)/2 or sat(left-right)
lpr_lmr	(left+right)/2 or sat(left+right)	(left-right)/2 or sat(left-right)
lmr_lpr	(left-right)/2 or sat(left-right)	(left+right)/2 or sat(left+right)

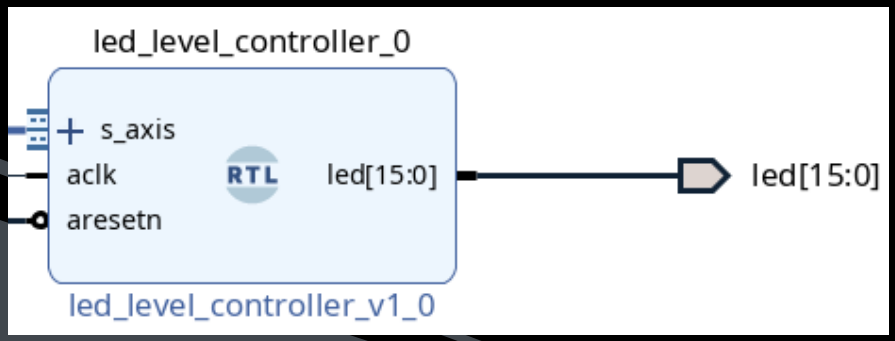
IP Path 2: OUTPUT SELECTOR 2)

The value of the led_r/g/b ports change as function of the status



Status	led_r	led_g	led_b
l_r	0xFF	0xFF	0xFF
mute	0xFF	0xFF	0x00
l_l	0xFF	0x00	0xFF
r_r	0xFF	0x00	0x00
lpr_lpr	0x00	0xFF	0xFF
lmr_lmr	0x00	0xFF	0x00
lpr_lmr	0x00	0x00	0xFF
lmr_lpr	0x00	0x00	0x00

IP Path 2: LED LEVEL CONTROLLER



LED level controller: it turns on the 16 LEDs on the board, depending on the level of the audio at the output (right and left channels are averaged); low/no audio ($<2^8$) no LEDs, saturation (2^{24}) all 16 LEDs.

e.g LOG VERSION

LOW OR NO AUDIO ($<2^8$)



MEDIUM LEVEL ($<2^{16}$)

BASYS 3 LEDS



UNTILL SATURATION ($<2^{24}$)



*LINEAR VERSIONS (as .bit provided) are accepted as well

UART

Work with 2000000 baudrate and set a clock UART frequency multiple of 32 MHz. 64 MHz, for example, will work fine.

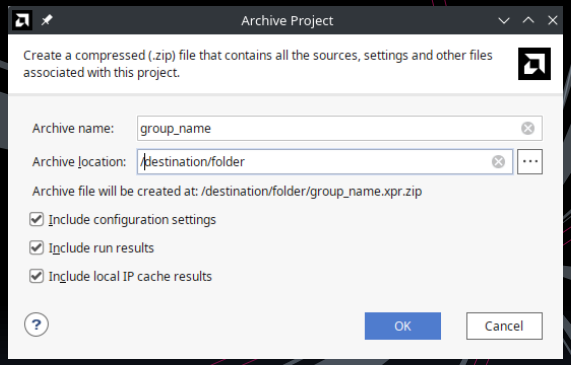
FSM in AXIS

You will not need to perform one AXI4-Stream transaction per clock cycle (AXIS is has way larger bandwidth than audio data)
Hence, you can use the AXIS FSM seen in the UART lesson, to ease the AXI handshake compliance, with the number of states that you need for each module.

AXIS handling is evaluated on its own, since you will likely reuse it in many modules.

Vivado archived project

You are requested to deliver your functional .zip project from Vivado, with your HDL modules inside.
In order to properly generate the archived project you need to go under
File→ Project → Archive
Then, flag all the options, rename the project with your group name, and click “ok”.



The output is the archived project you will upload on Piazza

Reports

POWER
Estimate the static power, the dynamic power and the temperature of the package.
Please, write an half page report.

TIMING
Detect the critical path for both setup and hold timing analysis.
Considering these information estimate the maximum clock frequency.
Please, write an half page report.

Maximum Frequency

If you implement pipelines correctly inside your HDL files, your project should achieve a maximum clock frequency of 200 MHz without timing errors.

- A must is to achieve the 100 MHz (0 points if < 100 MHz)
- 100 MHz < Frequency < 150 MHz is evaluated with 0.5 points
- Frequency ≥ 150 MHz is evaluated with 1 points.

Python script

You will be given a Python3 script that will allow you to test the acquisition of path 1, especially the FLOAT COMPRESSOR.

In order to use it:
- open a terminal in the script folder,
- execute the compiled file as follows, and specify with -t the duration of acquisition in seconds.

```
nicola@nicola-pc-lab:~/Downloads/python$ ./record_audio -t 100
```

The output will be a .wav file of the selected duration with left and right channel decompressed (FLOAT COMPRESSOR^{-1})

Valutazione

Project deliverables	points
7 SEGEMENT CONTROLLER	**0.5**
FLOAT COMPRESSOR (w/o generic)	**2.5**
GENERIC in FLOAT COMPRESSOR	**0.5**
SEND CONTROLLER	**1**
DEPACKETIZER	**1.5**
DIGILENT JSTK2	**1.5**
EFFECT SELECTOR	**0.5**
REVERBER	**2**
MOVING AVERAGE	**2**
OUTPUT SELECTOR	**2**
VOLUME	**2**
BALANCE	**1**
LED LEVEL CONTROLLER	**1**
REPORTS	**1**
CLOCK FREQUENCY	**1**
AXIS	**2**
Bonus (BINARY TO DECIMAL)	1